

Matrix-Free Gauge-Equivariant Neural Preconditioners for the Wilson–Dirac Operator

Adam Shaw¹ Yixuan Sun^{1,†} Srinivas Eswar¹ Xiaoye Li³ Yang Liu³ Phiala Shanahan² William Detmold²

¹Argonne National Laboratory · ²Massachusetts Institute of Technology · ³Lawrence Berkeley National Laboratory · [†]Corresponding author

Introduction

Solving linear systems $Ax = b$ is central to **lattice quantum chromodynamics (QCD)** [1]. These systems are large and ill-conditioned, so solvers such as **Conjugate Gradient (CG)** [2] are slow and expensive. A **preconditioner** M^{-1} is applied to both sides to lower the condition number:

$$M^{-1}Ax = M^{-1}b$$

Prior approaches fall short

Where they break

Many learned preconditioners store the full matrices A or M^{-1} [3, 4], or need to be retrained for each lattice size [5].

Both are impractical at production scale.

Adaptive multigrid (AMG) SOTA

The SOTA preconditioner avoids both problems, but carries a significant **setup cost** repeated for every new matrix A .

Our approach

We propose a framework, inspired by the Dirac operator’s dependency on gauge fields, that produces new configurations to generate preconditioners. Our framework is

Matrix-free

uses only $v \mapsto M^{-1}v$

Size-agnostic

no retraining between sizes

Lower setup cost

per gauge field (matrix A)

Lattice Gauge-Equivariant CNN

■ Gauge links $\in \text{SU}(3)$ ■ Gauge-equivariant layer ■ Our contribution

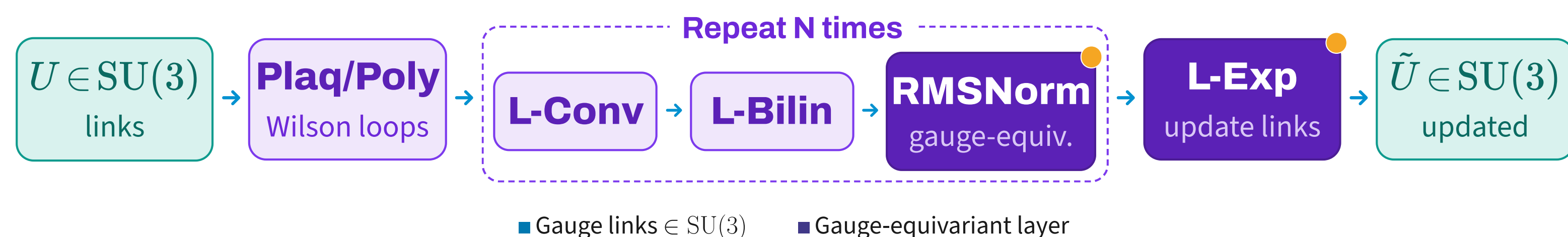


Figure 1. Part of an L-CNN, adapted from [7]. The original paper uses Plaques/Poly + L-Conv + L-Bilin + Trace + conventional CNN.

Background

L-CNNs **build gauge equivariance into the architecture** rather than learning it, letting them construct arbitrary Wilson loops and approximate any gauge-equivariant function [7].

- **Plaques/Poly** links $\rightarrow$ plaquettes (unit loops) & Polyakov loops (wrap periodic boundary).
- **L-Conv** a convolution that parallel-transport neighbors along the coordinate axes, creating arbitrary paths.
- **L-Bilin** a bilinear layer concatenating PT paths from the L-Conv. Stacking L-Conv + L-Bilin grows Wilson loops of arbitrary shape and size.
- **L-Exp** exponentiates the traceless anti-Hermitian part of the activations to update links, guaranteeing $\tilde{U} \in \text{SU}(3)$. Proposed, but unused, in [7].

Our work

Previous Argonne work [8] feeds links through a conventional CNN and into a preconditioner, but their CNN outputs arbitrary $\tilde{U}_\mu(x) \in \mathbb{C}^{3 \times 3}$, whereas operators such as $M^{-1} = D^\dagger D[\tilde{U}]$ expect $\tilde{U}_\mu(x) \in \text{SU}(3)$.

★ **Physically-grounded map.** We use **L-Exp** (proposed in [7], never used) to generate valid, physically-grounded $\tilde{U}_\mu(x) \in \text{SU}(3)$.

★ **Stable training.** A gauge-equivariant **RMSNorm** after each L-CB stabilizes training and prevents exploding activations.

Results

Preconditioner	Iters	Setup (GFLOP)
6× L-CNN $\rightarrow$ 6× Pfahler ★ OUR BEST	31	22
3× L-CNN $\rightarrow$ 3× Pfahler	34	12
1× L-CNN $\rightarrow$ 3× Pfahler	41	5
Identity $\rightarrow$ 3× Pfahler Best of [9]	48	0
AMG SOTA	60	71
6× L-CNN $\rightarrow (I+H)$	64	22
Identity $\rightarrow (I+H)$	68	0
3-Head CNN $\rightarrow (I+H)$ Best of [8]	86	2
Unpreconditioned	139	0

Table 1. PCG iterations to convergence (fewer is better) and per-gauge-field setup cost (billions of FLOPs), best to worst. Averaged over 20 gauge configurations U and, for each, 20 right-hand sides b , at lattice size $L = 8$.

48% fewer PCG iterations to convergence than AMG, the current state of the art.

Our best preconditioner, **L-CNN $\rightarrow$ PTLN**, chains the two networks:

$$\tilde{U} = \text{L-CNN}(U), \quad M^{-1}b = \text{PTLN}(\tilde{U}, b)$$

Methods

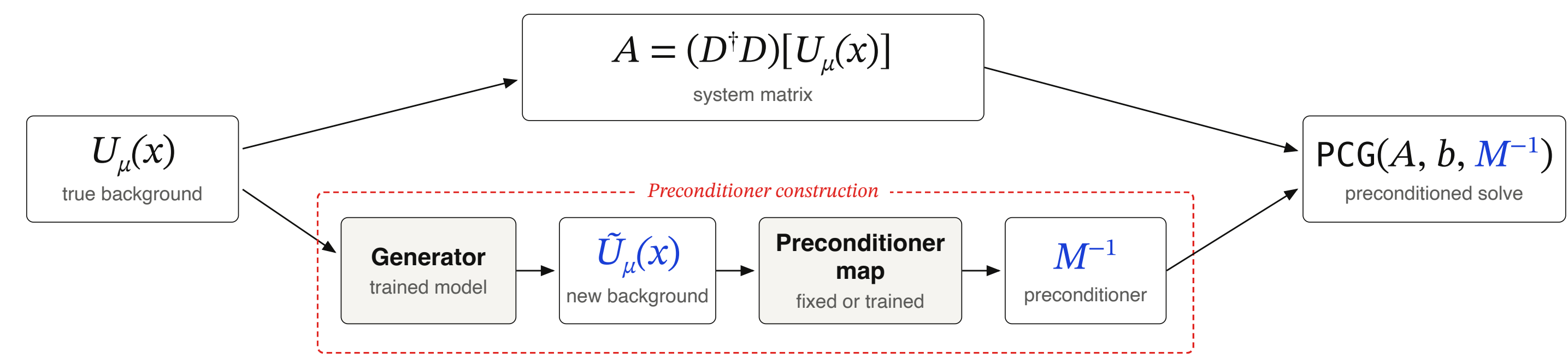
Setup

- 3 + 1 spacetime lattice.
- Complex gauge links $U_\mu(x) \in \text{SU}(3)$.
- $\kappa = 0.276, \beta = 5.5$.
- **Preconditioned Conjugate Gradient (PCG)** iterative solver [2].

A solvable system for PCG

PCG requires A and M^{-1} to be: **Fixed** **Linear** **HPD**. We build A from the discretized **Wilson–Dirac operator** $D[U_\mu(x)]$, solving the normal equations $D^\dagger D x = D^\dagger b$.

Our preconditioner



Training loss: unrolled PCG

We run k steps of PCG, set loss = $\|r_k\|$, **unroll the k steps**, and backpropagate through the network to update its weights.

Evaluation metric

A single metric, **iterations**: the number of PCG steps to convergence.

Parallel Transport Linear Network

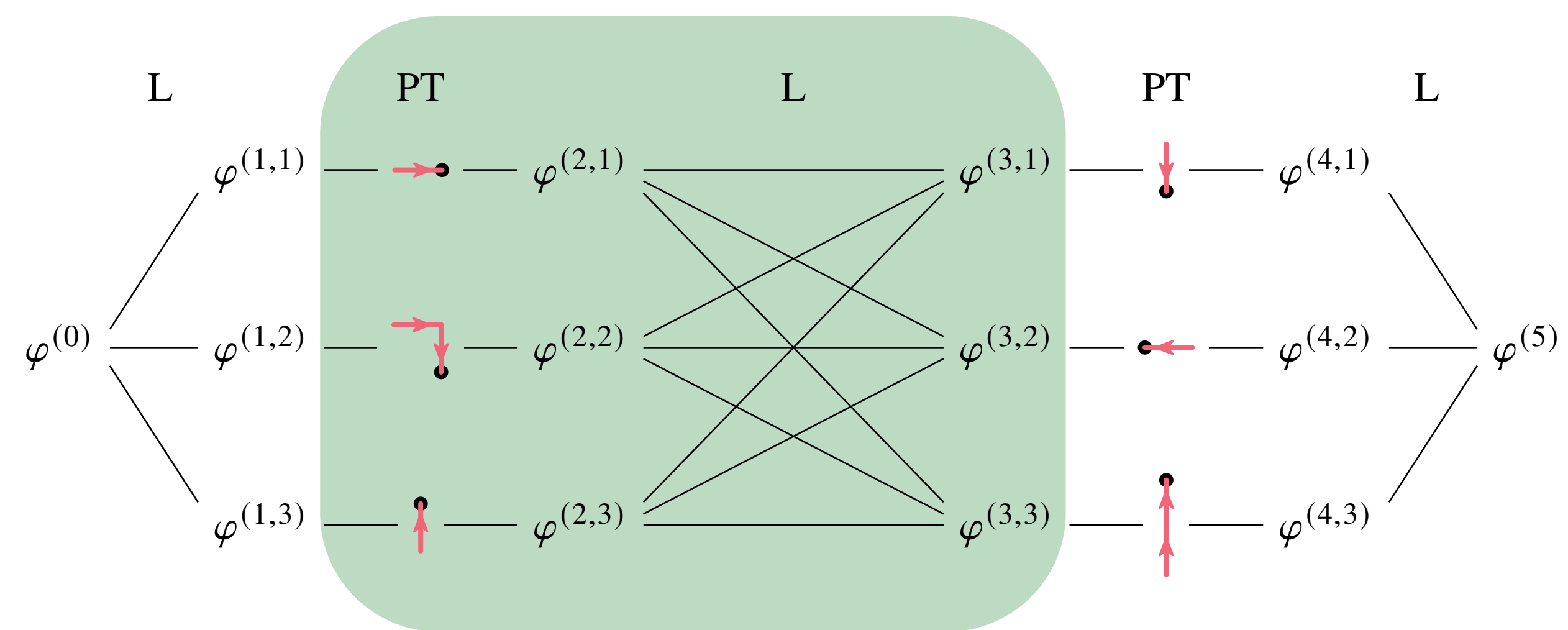


Figure 2. The parallel transport linear network (PTLN), adapted from Pfahler et al. [9]. Gauge-equivariant (more efficient) and linear (works with CG).

How it works

The PTLN [9] alternates two gauge-equivariant blocks acting on quark fields φ : a **parallel-transport layer (PT)** and a **linear layer (L)**.

- **PT layer** applies $T_p = H_{p_1} \cdots H_{p_\ell}$ (a chain of hops along a path p). The only layer that moves information across the lattice.
- **L layer** forms learnable combinations $\psi_b = \sum_a W_{ba} \varphi_a$, with $W_{ba} \in \mathbb{C}^{4 \times 4}$ on spinor indices. The only trainable parameters.
- **Fixed paths** the PT paths are hardcoded (chosen by us), not learned; all straight paths of length $\ell \in \{2^0, 2^1, \dots, 2^{\log L}\}$.

★ **Gauge-equivariant & linear.** Every layer is linear in φ (needed to sit inside CG) and gauge equivariant (shrinks the function space, boosts efficiency).

★ **It is the preconditioner.** We pass in a vector b ; the output is $\varphi^{(5)} \approx M^{-1}b$. No matrix is ever formed — only PT hops and small 4×4 spinor mixings.

Next Steps

- 1 Swap PCG for the **GCR solver**.
- 2 Explore **learning the parallel-transport paths** inside the PTLN.
- 3 Apply to pure gauge fields generated **without rectangle loops**.
- 4 Construct a comprehensive evaluation pipeline that also accounts for **solve wall-clock time** and at comparable **resource requirements**.
- 5 Test the framework extensively on **larger lattice sizes**.

References

- [1] Wilson, K. G. (1974). Confinement of quarks. *Phys. Rev. D*, 10, 2445–2459.
- [2] Saad, Y. (2003). *Iterative methods for sparse linear systems* (2nd edn).
- [3] Cali, S., Hackett, D. C., Lin, Y., Shanahan, P. E., & Xiao, B. (2023). Neural-network preconditioners for the Dirac equation. *Phys. Rev. D*, 107, 034508.
- [4] Sappl, J., Seiler, L., Harders, M., & Rauch, W. (2019). Deep learning of preconditioners for conjugate gradient solvers. arXiv:1906.06925.
- [5] Häusner, P., Öktem, O., & Sjölund, J. (2023). Neural incomplete factorization: learning preconditioners for CG. arXiv:2305.16368.
- [6] Clark, M. A., Babich, R., Barros, K., Brower, R. C., & Rebbi, C. (2010). Solving lattice QCD systems using mixed-precision solvers on GPUs. *Comput. Phys. Commun.*, 181, 1517–1528.
- [7] Favoni, M., Ipp, A., Müller, D. I., & Schuh, D. (2022). Lattice Gauge Equivariant Convolutional Neural Networks. *Phys. Rev. Lett.*, 128, 032003.
- [8] Sun, Y., Eswar, S., Lin, Y., Detmold, W., Shanahan, P., Li, X., Liu, Y., & Balaprakash, P. (2025). Matrix-free neural preconditioner for the Dirac operator. arXiv:2509.10378. *ACM TAIS (in press)*.
- [9] Pfahler, S., Knüttel, D., Lehner, C., & Wettig, T. (2026). A novel gauge-equivariant neural-network architecture for preconditioners in lattice QCD. arXiv:2602.23840.